

1. Introduction

This report documents the design, implementation, and evaluation of Activity 7 - the simulation-based Decision Support System (DSS) extension of the Erreka maintenance platform developed throughout the course.

The core objective of Activity 7 is to extend the existing Erreka DSS into a forward-looking simulation engine that lets a maintenance planner compare three distinct maintenance strategies over time, observe the trade-offs between failure rates, SLA breaches, and maintenance cost, and receive a concrete recommendation about which strategy best suits their operational context.

The key distinction from previous activities is the shift from observation to projection - from describing what has happened to exploring what would happen under different decision policies. This shift represents the prescriptive layer of the DSS, which was absent from prior activities.

2. System Overview

2.1 Architecture

The system is structured into four independent components:

- simulation/ - Python script (erreka_simulation.py) that runs the Markov-chain simulation and writes the results to disk
- data/ - Input CSV (door_risk_clasification.csv) generated in Activity 5, containing 504 doors with risk scores, SLA categories, usage scenarios, and failure histories
- output/ - Generated JSON and CSV files consumed by the web interface
- interface/ - Static HTML/CSS/JS application that reads the output files and renders the full DSS interaction flow

2.2 User Flow

The application exposes four navigation sections: Home, Simulate, Compare, and Results. The intended user journey follows a deliberate sequence:

- Home - overview of the installed base (504 doors, 76 high-risk, 998 total incidents)
- Simulate - user selects which strategies to compare and the time horizon (3, 6, or 12 months)
- Simulate (Strategy Rules) - explanatory table showing trigger logic, inspection frequency, repair speed, and cost profile for each strategy
- Simulate (Engine Flow) - diagram exposing the monthly simulation loop so the user understands the model before seeing results
- Simulate (State Transitions) - the Markov diagram and probability table, with explicit disclosure that values are modelling assumptions
- Results (Single Strategy View) - deep-dive charts per strategy before side-by-side comparison
- Compare - all three strategies on the same axes; comparison table with best-value highlights
- Results (Decision Summary) - dynamic text recommendation and top-5 priority door list, filterable by strategy

The flow is intentionally linear and progressive: the user cannot reach the results without first having been shown the model assumptions. This design choice is deliberate - a DSS that

presents numbers without explaining how they were computed cannot support genuine decision-making.

3. Simulation Model

3.1 State Space

Each of the 504 doors is modelled as a discrete-state system with three possible states at any given time step:

- OK - the door is operational and not showing signs of wear
- Degrading - the door is operational but exhibiting signs of wear; intervention can prevent failure
- Failed - the door is non-functional; repair is required

There is intentionally no direct OK → Failed transition. This reflects the assumption that real industrial doors do not fail instantaneously; they degrade first, and the degradation window is the point at which the maintenance strategy can intervene. A Repaired pseudo-state is used internally to signal that a failed door has been fixed and will return to OK on the next time step.

3.2 Initial State Assignment

Door initial states are derived directly from the Activity 5 risk classification, creating a meaningful link between the predictive and simulation layers:

- High risk (score = 3) → initial state: Degrading
- Medium risk (score = 2) → initial state: Degrading (50%) or OK (50%) - stochastic
- Low risk (score = 1) → initial state: OK

This ensures the simulation starts from a realistic distribution rather than an arbitrary all-OK baseline. The random seed is fixed (RANDOM_SEED = 42) for full reproducibility.

3.3 Time Horizon and Granularity

The simulation runs at monthly time steps over a maximum horizon of 12 months. The Python engine always computes all 12 months; the interface slices the results to the user-selected horizon (3, 6, or 12 months). This means switching the displayed horizon requires no re-computation.

At each monthly time step the engine: (1) iterates through all 504 doors, (2) reads each door's current state, (3) applies the active strategy's transition probabilities - adjusted for door-level context - (4) updates the state stochastically, (5) collects metrics, and (6) stores results for that month.

3.4 Transition Probabilities

Transition probabilities are the core parameters of the simulation. They are modelling assumptions justified by the operational logic of each strategy, not values derived from historical Erreka failure data. This is explicitly stated in the interface on every screen where probabilities appear.

Transition	Reactive	Preventive	Risk & SLA
OK → Degrading	0.15	0.08	0.05
Degrading → Failed	0.40	0.20	0.10
Failed → Repaired	0.60	0.80	0.95

Table 1. Baseline transition probabilities per strategy

The Risk & SLA strategy additionally applies door-level adjustments: for doors with Critical SLA and $\text{risk_score} \geq 2$, the Degrading → Failed probability is reduced by a further 30% and the repair probability is increased. For very high criticality doors, the OK → Degrading probability is reduced by 20%. These adjustments reflect the targeted, context-aware nature of a risk-based strategy.

3.5 Cost Model

Maintenance cost is computed at each time step based on the actions triggered by the active strategy. The cost model distinguishes three action types:

Action	Reactive (€)	Preventive (€)	Risk & SLA (€)
Repair (after failure)	850	500	600
Inspection	0	120	150
Targeted preventive visit	0	200	350

Table 2. Cost per maintenance action by strategy

SLA breach penalties are added on top of repair costs whenever a Critical or High Priority door fails. These are: Critical SLA - €2,000 per breach; High Priority SLA - €1,000 per breach; Standard SLA - €300 per breach. A door that remains Failed in the following month incurs an additional 50% of the Critical SLA penalty if it carries a Critical SLA classification.

4. Maintenance Strategies

4.1 Reactive-Only Strategy

The Reactive strategy represents the baseline: no planned maintenance takes place, and the team only intervenes after a failure has occurred. It has the highest individual repair cost (€850 per incident) and the highest transition probabilities, reflecting the absence of preventive intervention. Inspection cost is zero because no inspections are scheduled. This strategy is the implicit default that most maintenance teams operate under before structured planning is introduced, and it serves as the comparison baseline throughout the DSS.

4.2 Preventive Strategy (Usage-Based)

The Preventive strategy schedules monthly inspections for all 504 doors regardless of their individual risk profile. This blanket approach reduces degradation and failure probabilities significantly compared to Reactive (OK → Degrading drops from 0.15 to 0.08, Degrading → Failed from 0.40 to 0.20), but it incurs a high fixed monthly cost because every door is inspected every month. For doors with intensive usage scenarios, the OK → Degrading probability is slightly increased ($\times 1.2$) to reflect higher wear rates. The strategy's weakness is a lack of targeting: it spends the same resources on a low-risk door as on a high-risk one.

4.3 Risk- and SLA-Based Strategy

The Risk & SLA strategy applies targeted interventions: only doors with $\text{risk_score} \geq 2$ receive preventive visits (at €350 per visit), which is a subset of the installed base. In exchange, it achieves the lowest failure probabilities (0.05 and 0.10 for OK → Degrading and Degrading → Failed respectively) and the highest repair probability (0.95), because critical doors are prioritised. The strategy's strength is efficiency: it concentrates resources where the business risk is highest. Its weakness is that low-risk doors receive no proactive attention at all.

5. Simulation Results

5.1 12-Month Summary

The following table summarises the simulation results over the full 12-month horizon, using the fixed random seed for reproducibility.

Strategy	Total Failures	SLA Breaches	Total Cost (€)	Avg Failures/mo
Reactive	568	465	€1,167,800	47.3
Preventive	294	237	€1,253,040	24.5
Risk & SLA	144	102	€813,150	12.0

Table 3. 12-month simulation totals. Green cells indicate best value per metric.

Risk & SLA achieves the best outcome on all four metrics simultaneously. Compared to the Reactive baseline, it reduces total failures by 74.6%, SLA breaches by 78.1%, and total cost by 30.3%. Preventive reduces failures and breaches meaningfully compared to Reactive but ends up as the most expensive strategy overall due to the cost of inspecting all 504 doors every month regardless of their risk level.

5.2 Monthly Trajectory

The monthly data reveals patterns that aggregate totals conceal. Reactive failures grow over time (from 31 in Month 1 to 66 in Month 12) as unrepaired or slowly-repaired doors accumulate in degraded states. Preventive stabilises quickly and remains roughly flat across all 12 months, reflecting the constant inspection cadence. Risk & SLA achieves low and stable failure counts for most months but shows occasional spikes (Month 10: 23 failures) - a consequence of its targeted approach leaving some non-critical doors unmonitored.

These trajectory differences are important for decision-making: a strategy that starts well but trends upward (Reactive) represents a different operational risk from one that starts modestly but stabilises (Preventive), even if their totals at a given point are similar.

6. DSS Value and Decision Support

The primary user of this system is a maintenance planner at Erreka - someone responsible for allocating the maintenance budget across an installed base of 504 doors in multiple countries. Prior to this activity, the planner had access to descriptive analytics (Activity 4: what failures happened, where) and predictive risk classification (Activity 5: which doors are most likely to fail next). Neither layer answered the strategic question: given everything we know, which maintenance policy should we actually adopt?

That question is prescriptive. It requires projecting forward under different policies and evaluating the trade-offs between them. Until now, that decision would have been made on intuition and experience, without supporting numbers to justify or challenge the choice.

This system fills that gap. The planner can now take the same installed base and project it forward under three policies, see the concrete outcomes on failures, SLA breaches, and cost, and receive an automatically generated recommendation. Critically, every assumption in the model is disclosed - the strategy rules table, the engine flow diagram, and the transition probability table are all presented before the results, so the planner can challenge the model if the assumptions do not reflect their operational reality.

The Decision Summary screen adds a final layer of operational value: the top-5 priority door list, which translates the aggregate simulation results into a concrete list of doors to intervene on first. The list is strategy-aware - Reactive ranks by current failure state, Preventive by usage intensity among operational doors, Risk & SLA by business impact score - so the planner gets an actionable output regardless of which strategy they choose.

7. Critical Reflection

7.1 Model Limitations

The most significant limitation of the current model is that the transition probabilities are assumptions, not values learned from Erreka's historical failure data. While the assumed values are internally consistent and directionally plausible, the absolute numbers produced by the simulation (total failures, total cost) should not be treated as predictions. They are useful for comparing strategies against each other, not for forecasting actual operational outcomes.

A second limitation is that the simulation operates on the aggregate installed base without dynamic filtering. The filter controls (door type, country, criticality level) are present in the UI but not yet wired to the simulation engine. As a consequence, the current version cannot answer questions such as 'what is the best strategy for ROLLfast doors in countries with Critical SLAs?' - a query that would be highly relevant in practice.

A third limitation is the simplified state space. Real doors may exhibit more nuanced degradation patterns, seasonal effects, or dependencies between nearby units. The three-state Markov model abstracts away all of this in favour of computational simplicity.

7.2 Sensitivity to Assumptions

The simulation is sensitive to the transition probability values, particularly the Degrading → Failed probability. A small change in this parameter for the Reactive strategy - for example, increasing it from 0.40 to 0.50 - would substantially increase its total failure count and cost, making the case for Risk & SLA even stronger. Conversely, reducing the Preventive inspection cost assumption would erode the cost advantage of Risk & SLA.

The cost model is also assumption-dependent. The SLA breach penalties (€2,000 for Critical, €1,000 for High Priority) are plausible but unverified. In a real deployment, these values would be negotiated SLA contract terms, not assumptions.

Because of this sensitivity, the system is designed to support comparison rather than prediction. The value lies in showing that Risk & SLA dominates Reactive under a wide range of reasonable assumptions, not in the precise magnitude of the differences.

7.3 Applicability in Real Decision-Making

The system is applicable as a structured discussion tool rather than a black-box decision engine. It gives maintenance planners and management a shared quantitative framework for a conversation that would otherwise happen on intuition alone. The explicit disclosure of assumptions is essential to this role: if the planner disagrees with a probability value, they can adjust it (currently by editing the Python config) and re-run, which turns the model into an interactive what-if tool rather than a fixed oracle.

The natural next step towards production applicability is recalibrating the transition probabilities against Erreka's historical failure data. The architecture supports this: `TRANSITION_PROBS` is a single dictionary in the simulation config, and replacing assumed values with fitted values requires no structural change. Once calibrated, the system would move from plausible to empirically grounded, and the absolute numbers would carry genuine predictive value alongside the comparative value the current version already provides.

8. Conclusion

Activity 7 extends the Erreka DSS from a descriptive and predictive platform into a simulation-based decision support system capable of comparing maintenance strategies, projecting operational outcomes over time, and generating actionable recommendations. The system implements a three-state Markov simulation engine across 504 doors, compares Reactive, Preventive, and Risk & SLA strategies on failures, SLA breaches, and cost, and presents results through a progressive, assumption-transparent interface.

In general, even though this activity really was by far the hardest one so far, but I enjoyed doing it because I love doing web pages and I love doing in HTML and CSS (JS a bit less), and I was excited to visualize with these tools all my work so far and connect it with databases with we have been working whole semester!